

Chapter 55: Large-Scale Folder Structures

1. Overview

Angular does not enforce a folder structure. The CLI scaffolds a flat `src/app` with components sitting next to modules, and that is fine for a todo app. It is not fine for an application with 40 developers, 200 routes, and an 18-month roadmap. At that scale, folder structure stops being a cosmetic preference and becomes an architectural decision with the same weight as choosing NgRx vs. signals for state management. A bad structure doesn't crash the app — it silently taxes every future pull request with longer code-review time, harder-to-find files, accidental cross-feature coupling, and slower builds.

This chapter covers the two dominant organizational philosophies (type-based vs. feature/domain-based), the `core/shared/features` convention that most enterprise Angular codebases converge on, the barrel file (`index.ts`) debate — including the very real tree-shaking and circular-dependency traps it introduces — colocation of tests/styles/component files, the idea of a feature's "public API surface," and how to scale a structure as an app grows from a weekend project to a multi-team enterprise monorepo. We finish with two full example trees (mid-size and large/Nx-style) and a substantial interview Q&A section, because "how do you structure a large Angular app" is one of the most common staff/principal-level interview questions — it's a proxy for "have you actually maintained something big, or only built demos."

2. Core Concepts

2.1 Type-Based ("Layer-First") Organization

This is the structure most tutorials teach and most small apps start with:

```
src/app/  
  components/  
    header.component.ts  
    footer.component.ts  
    product-card.component.ts  
    user-avatar.component.ts  
  services/  
    auth.service.ts  
    product.service.ts  
    cart.service.ts  
  pipes/  
  directives/  
  models/  
  guards/
```

Everything of the same *kind* lives together, regardless of what feature it belongs to. This mirrors classic MVC folder conventions (`controllers/`, `models/`, `views/`) from pre-SPA web frameworks.

Why it fails at scale:

- To understand or modify "checkout," you must jump between `components/checkout-*.ts`, `services/checkout.service.ts`, `models/checkout.model.ts`, `guards/checkout.guard.ts` — five folders for one feature.

- Folders balloon into hundreds of unrelated files (`components/` with 150 entries) with no natural subgrouping, so IDE fuzzy-find becomes the only navigation method.
- There is no folder-level signal for feature ownership, so it's impossible to say "team A owns this directory" — file-level CODEOWNERS entries become unmanageable.
- Deleting a feature means hunting across five top-level folders to remove every trace; it's very easy to leave orphaned services behind.

Type-based organization is acceptable, even preferable, for **small apps** (roughly under ~15-20 components) where the overhead of feature folders isn't justified yet. It becomes a liability the moment a second developer needs to work in a feature they didn't write.

2.2 Feature-Based / Domain-Based Organization

Instead of grouping by technical kind, group by **business capability** — everything related to "orders" lives under `features/orders/`, regardless of whether it's a component, service, model, or test:

```
src/app/features/orders/
  order-list/
    order-list.component.ts
    order-list.component.html
    order-list.component.scss
    order-list.component.spec.ts
  order-detail/
    order-detail.component.ts
  ...
  data-access/
    order.service.ts
    order.model.ts
  order.routes.ts
```

Why it wins at scale:

- Cognitive locality: everything needed to understand or change "orders" is under one directory. A new hire can be handed `features/orders/` and be productive without reading the rest of the app.
- Deletability: removing a feature is (ideally) `rm -rf features/orders` plus removing one route registration — a genuine architectural test of good boundaries ("can I delete this folder and only break one import line?").
- Natural alignment with lazy-loaded route boundaries (see §4) — one feature folder maps to one lazily-loaded chunk.
- Natural alignment with team ownership — CODEOWNERS, PR routing, and Nx/module-boundary lint rules can all key off top-level feature folder names.
- Scales horizontally: adding feature #40 doesn't make feature #1 harder to find, unlike type-based folders which get uniformly worse as the app grows.

This is why virtually every serious Angular style guide (the official Angular style guide's "LIFT" principle — Locate, Identify, Flat, Try to be DRY — and community guides like Nrwl's Nx workspace conventions) push toward feature-based, or a hybrid of feature-based with a few shared type-based buckets (`core/` , `shared/`).

2.3 The `core / shared / features` Convention

The most widely adopted enterprise Angular convention (originating from the official Angular docs' now-retired style guide, still followed almost universally) splits the app into three top-level buckets:

```
src/app/  
  core/      <- singleton services, app-wide config, one-time initializers  
  shared/    <- reusable, presentation-only, no business logic tied to one feature  
  features/  <- one folder per business domain/route area
```

`core/` — things instantiated exactly once for the lifetime of the app:

- `AuthService`, `HttpErrorInterceptor`, `LoggingService`, app initializers, root-level guards.
- Imported once, in `app.config.ts` / `AppModule`. Never imported by a feature module directly beyond DI — features consume `core` services via injection, not by importing `core`'s internal files.
- Rule of thumb: if this file must never have two live instances, it's `core`. A guard placed in `core` to protect a route enforces this pattern; some teams add a lint rule (or a runtime check in `CoreModule.forRoot()`, pre-standalone) that throws if `CoreModule` is imported more than once.

`shared/` — dumb, reusable, feature-agnostic building blocks:

- Generic UI components (`ButtonComponent`, `ModalComponent`, `DataTableComponent`), pipes (`TruncatePipe`), directives (`ClickOutsideDirective`), and pure utility functions.
- Should have **zero knowledge of any specific feature's domain model**. A `shared` component takes primitives or generic `@Input()`s — it does not import `Order` or `Invoice` types from a feature.
- Can be imported by any feature, and by `core` in some architectures (rare) — but `shared` must never import from `features/*`. That import direction is a one-way street; violating it is one of the most common architecture-review findings in large Angular codebases.

`features/` — one folder per domain area (`orders/`, `billing/`, `dashboard/`), each self-contained, each lazy-loadable, each ideally unaware of its sibling features' internals.

A simplified dependency graph:

```
features/orders ---> shared, core (via DI)  
features/billing ---> shared, core (via DI)  
shared          ---> (nothing feature-specific)  
core            ---> (nothing feature-specific)
```

Arrows only point "inward/upward." A `shared` component depending on a `features/orders` type is a smell that signals the component isn't actually generic and should either move into `features/orders` or be genuinely generalized.

2.4 Colocating Tests, Styles, and Component Files

The Angular CLI default — `component.ts`, `.html`, `.scss`, `.spec.ts` sitting in the same folder — is itself a colocation decision, and it's the right one. The alternative (a top-level `tests/` mirroring the `src/` tree, common in older Java/.NET-influenced codebases) actively hurts:

- Renaming or moving a component means touching two parallel directory trees that must stay in sync.
- Deleting a component is easy to do incompletely — the orphaned spec file lingers, silently passing (or worse, causing a stale build failure) with no visual proximity to notice it.
- Reviewers see the implementation and its test in the same diff hunk grouping in most IDEs and PR tools, improving review quality.

The same argument extends to styles: a component's `.scss` colocated next to its `.ts` file makes `viewEncapsulation` scoping intuitive — you're always looking at "this component's own styles," not hunting through a global `styles/` tree trying to figure out which selectors apply where. Global/shared SCSS (variables, mixins, resets) still belongs in a top-level `styles/` or `shared/styles/` folder — colocation applies to component-specific concerns, not truly global ones.

Some teams also colocate a component's Storybook story (`*.stories.ts`) and state (`*.store.ts` for a component-scoped signal store) in the same folder for the same reason: one folder = one unit of understanding.

2.5 Public API Surface Per Feature

A feature folder should expose a deliberately small, curated **public API** — the route configuration (or a top-level entry component) and perhaps a couple of exported types other features are permitted to depend on — while hiding its internals (child components, internal services, internal state stores) from the rest of the app.

Concretely, this usually means:

```
features/orders/
  index.ts                <- public API: exports OrderRoutes, OrderSummary (a DTO type)
  order.routes.ts
  data-access/
    order.service.ts      <- NOT exported from index.ts - internal
    order.model.ts
  ui/
    order-list.component.ts <- NOT exported - internal
    order-detail.component.ts
```

Other features (or `app.routes.ts`) import **only** from `features/orders` (resolved to `features/orders/index.ts`), never reach into `features/orders/ui/order-list.component`. This is enforced two ways in practice:

1. **Convention + code review** — works for small teams, fails silently at scale because nothing stops a developer from doing a deep import when it's the path of least resistance under deadline pressure.
2. **Tooling enforcement** — Nx's `enforce-module-boundaries` ESLint rule (tagged modules, e.g. `scope:orders`, `type:feature`, `type:data-access`) or a plain ESLint `no-restricted-imports` rule that blocks any import matching `features/*/ui/**` or `features/*/data-access/**` from outside that feature. This is the only approach that actually holds at 30+ engineers — convention alone degrades.

This "public API surface" idea is exactly the module-boundary concept from Nx's "buildable/publishable library" model, and it's the folder-structure analogue of encapsulation in OOP: hide implementation, expose contract.

2.6 Scaling the Structure as the App Grows

Stage	App size	Recommended structure
Prototype	< 10 components, 1 dev	Flat <code>src/app/</code> , type-based is fine, don't over-engineer
Small app	10–30 components, 1–3 devs	Introduce <code>core/</code> and <code>shared/</code> ; features still fairly flat
Mid-size app	30–150 components, one team	Full <code>core/shared/features</code> with lazy-loaded feature routes; barrel files per feature for a clean public API
Large enterprise app	150+ components, multiple teams	Nx (or similar) monorepo: <code>apps/</code> + <code>libs/</code> split by <code>scope</code> (domain) and <code>type</code> (feature/ui/data-access/util), enforced module boundaries, generated dependency graphs

The key insight interviewers are probing for: **structure should track the current pain, not a hypothetical future one.** Introducing an Nx monorepo with 8 library types for a 5-screen MVP is over-engineering; keeping a 300-component app in one flat `components/` folder is under-engineering. The refactor from mid-size to large-enterprise structure is itself a milestone most staff engineers will be asked to describe candidly — "we outgrew `shared/`, here's how we split it" is a legitimate, expected interview narrative.

3. Code Examples

3.1 Full Realistic Mid-Size App Structure

A single-team SaaS admin dashboard, ~60 components, standalone-components era Angular (v17+), still a single deployable app:

```
src/  
  app/  
    app.config.ts  
    app.routes.ts  
    app.component.ts  
  
  core/  
    auth/  
      auth.service.ts  
      auth.guard.ts  
      auth.interceptor.ts  
      token-storage.service.ts  
    layout/  
      app-shell.component.ts  
      app-shell.component.html  
    error-handling/  
      global-error-handler.ts  
      http-error.interceptor.ts  
    core.providers.ts      <- provideCore() helper aggregating core providers
```

```

shared/
  ui/
    button/
      button.component.ts
      button.component.html
      button.component.scss
      button.component.spec.ts
    modal/
      modal.component.ts
      modal.component.spec.ts
    data-table/
      data-table.component.ts
      data-table.component.spec.ts
    pipes/
      truncate.pipe.ts
      currency-short.pipe.ts
    directives/
      click-outside.directive.ts
    utils/
      date.utils.ts
      form.utils.ts
    shared.index.ts          <- barrel: exports only intended-public shared items

features/
  dashboard/
    dashboard.routes.ts
    dashboard.component.ts
    widgets/
      revenue-widget.component.ts
      activity-widget.component.ts
    data-access/
      dashboard.service.ts
    index.ts

orders/
  order.routes.ts
  ui/
    order-list/
      order-list.component.ts
      order-list.component.html
      order-list.component.scss
      order-list.component.spec.ts
    order-detail/
      order-detail.component.ts
      order-detail.component.spec.ts
    order-status-badge/
      order-status-badge.component.ts
  data-access/
    order.service.ts
    order.model.ts
    order.store.ts          <- signal-based feature state
  index.ts                  <- exports ORDER_ROUTES + OrderSummary type only

```

```

    billing/
      billing.routes.ts
    ui/
      invoice-list/
      payment-method-form/
    data-access/
      billing.service.ts
      invoice.model.ts
    index.ts

  settings/
    settings.routes.ts
  ui/
    profile-form/
    notification-preferences/
  data-access/
    settings.service.ts
  index.ts

assets/
environments/
styles/
  _variables.scss
  _mixins.scss
  _reset.scss
  styles.scss                <- global entry, imports the above

```

`app.routes.ts` only ever imports each feature's top-level route array:

```

// app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'orders',
    loadChildren: () => import('./features/orders').then(m => m.ORDER_ROUTES),
  },
  {
    path: 'billing',
    loadChildren: () => import('./features/billing').then(m => m.BILLING_ROUTES),
  },
  {
    path: 'dashboard',
    loadChildren: () => import('./features/dashboard').then(m => m.DASHBOARD_ROUTES),
  },
];

```

3.2 Full Realistic Large Enterprise Structure (Nx-Style Monorepo)

A multi-team platform — e.g., an e-commerce company with separate teams for storefront, checkout, admin tooling, and a design system team — using Nx to split `apps/` (deployables) from `libs/` (everything shared/importable), tagged by `scope` and `type`:

```
apps/
  storefront/                <- deployable app: customer-facing site
    src/app/app.routes.ts
    src/app/app.config.ts
    project.json
  admin-console/             <- deployable app: internal admin tool
    src/app/app.routes.ts
    project.json
  storefront-e2e/
  admin-console-e2e/

libs/
  orders/
    feature-order-list/      <- type:feature, scope:orders
      src/lib/order-list.component.ts
      src/lib/order-list.routes.ts
      src/index.ts           <- public API barrel
      project.json           <- tags: ["scope:orders", "type:feature"]
    feature-order-detail/
    data-access-orders/      <- type:data-access, scope:orders
      src/lib/order.service.ts
      src/lib/order.model.ts
      src/lib/order.store.ts
      src/index.ts
      project.json           <- tags: ["scope:orders", "type:data-access"]
    ui-order-status-badge/   <- type:ui, scope:orders
      src/lib/order-status-badge.component.ts
      src/index.ts
      project.json           <- tags: ["scope:orders", "type:ui"]
    util-orders/             <- type:util, scope:orders
      src/lib/order-formatting.ts

  billing/
    feature-invoices/
    data-access-billing/
    ui-invoice-card/

  checkout/
    feature-cart/
    feature-payment/
    data-access-checkout/

  shared/
    ui-components/           <- type:ui, scope:shared – buttons, modals, tables
      src/lib/button/
      src/lib/modal/
```



```

    src/lib/data-table/
    util-formatting/          <- type:util, scope:shared – date/currency utils
    data-access-auth/         <- type:data-access, scope:shared – AuthService, guards
    design-tokens/           <- SCSS variables, theming

  platform/
    core-http/                <- HTTP interceptors, error handling, base API client
    core-logging/
    core-config/              <- environment/feature-flag access

  nx.json
  tsconfig.base.json          <- path mappings, e.g. "@myorg/orders/data-access"

```

`tsconfig.base.json` maps each library's public entry point:

```

{
  "compilerOptions": {
    "paths": {
      "@myorg/orders/feature-order-list": ["libs/orders/feature-order-list/src/index.ts"],
      "@myorg/orders/data-access": ["libs/orders/data-access-orders/src/index.ts"],
      "@myorg/shared/ui-components": ["libs/shared/ui-components/src/index.ts"]
    }
  }
}

```

Module boundaries are enforced with an ESLint rule keyed off tags:

```

// .eslintrc.json (relevant rule)
{
  "rules": {
    "@nx/enforce-module-boundaries": [
      "error",
      {
        "depConstraints": [
          { "sourceTag": "scope:orders", "onlyDependOnLibsWithTags": ["scope:orders",
"scope:shared", "scope:platform"] },
          { "sourceTag": "type:ui", "onlyDependOnLibsWithTags": ["type:util"] },
          { "sourceTag": "type:feature", "onlyDependOnLibsWithTags": ["type:ui", "type:data-
access", "type:util"] },
          { "sourceTag": "type:data-access", "onlyDependOnLibsWithTags": ["type:util",
"type:data-access"] }
        ]
      }
    ]
  }
}

```

This is the enforcement mechanism that makes "orders can't reach into checkout's internals" a **build-time lint failure**, not a code-review hope.

3.3 An Example Barrel File and Its Tradeoffs

```
// features/orders/index.ts
export { ORDER_ROUTES } from './order.routes';
export type { OrderSummary, OrderStatus } from './data-access/order.model';
// Deliberately NOT exported: OrderService, OrderStore, OrderListComponent,
// OrderDetailComponent – these are internal implementation details.
```

Consuming code:

```
// app.routes.ts
import { ORDER_ROUTES } from './features/orders'; // clean, stable import path
```

What this buys you:

- A single, stable import path (`'./features/orders'`) that doesn't change even if internal files are renamed, split, or reorganized — refactors inside the feature don't ripple outward.
- An explicit, reviewable "public API" — a PR that adds an export to `index.ts` is a visible signal "this is now part of the contract," which prompts more careful review than a deep import silently reaching into an internal file.
- Cleaner import statements for consumers (`import { Button, Modal } from '@shared/ui'` instead of three separate deep-path imports).

What it costs you (see §4 for the mechanics):

- **Tree-shaking degradation.** A barrel that re-exports 40 things from a `shared/ui` library means importing *one* component can pull the whole barrel's module graph into the compilation unit unless the bundler's side-effect analysis is airtight — in practice this has caused real, measured bundle-size regressions in large Angular/Nx codebases, which is why Nx's default generated libraries deliberately keep barrels small and `sideEffects: false` is set in `package.json` for buildable libraries.
- **Circular import risk.** If `features/orders/index.ts` re-exports something that transitively imports back into `features/billing/index.ts`, and `billing`'s barrel imports back into `orders`, you get a real circular dependency that TypeScript/webpack will sometimes tolerate (with values silently becoming `undefined` at runtime) and sometimes fail outright on, depending on evaluation order. Barrels make circularity *easy to introduce accidentally* because the aggregation hides the true import graph from the person adding one more export.
- **Slower incremental builds / IDE "go to definition."** Every consumer of a barrel is coupled to the entire barrel file for change-detection purposes in some build tools' caching — touching an unrelated export in the same `index.ts` can invalidate more downstream cache entries than a direct file import would.

The pragmatic rule most large teams land on: **use barrel files at the feature/library public-API boundary (one per library), never as a blanket practice inside every subfolder.** A barrel in `data-access/index.ts`, another in `ui/index.ts`, another in the feature root — three layers of barrels — is a common but avoidable source of the tree-shaking and circularity problems above.

4. Internal Working

4.1 Folder Structure and Lazy-Loading Boundaries

The Angular Router's `LoadChildren` (and `LoadComponent`) create genuine code-splitting boundaries at the bundler level. Whatever module graph is reachable from the dynamically-imported file becomes its own chunk. This means:

- If `features/orders/index.ts` is the `LoadChildren` target, **everything the barrel re-exports, and everything those exports transitively import, ends up eagerly bundled into the `orders` chunk** — even parts of `orders` never actually used by the routes that got loaded. A barrel that re-exports both `OrderListComponent` and a rarely-used `OrderBulkImportComponent` forces both into the same chunk unless the bulk-import component is itself behind its own `LoadComponent()` boundary.
- Well-designed feature folders keep the lazy-loading entry point (`*.routes.ts`) intentionally thin, deferring to further nested `LoadComponent()` calls for large sub-screens, so the initial feature chunk stays small and rarely-visited screens split further.
- This is why the "one lazy-loaded route per feature folder" convention isn't just organizational neatness — it's a direct, mechanical mapping to what actually ends up in a separate downloaded JS file. Reorganizing folders without updating `LoadChildren` boundaries accordingly can silently merge chunks that should have stayed separate, or vice versa — always check the production build's chunk output (`ng build --stats-json` + a bundle analyzer) after a structural refactor.

4.2 How Barrel Files Interact with Tree-Shaking

Tree-shaking (dead-code elimination performed by esbuild/webpack/Rollup during the production build) works by statically analyzing ES module `import/export` graphs and dropping exports nothing imports, **as long as it can prove doing so is safe** — i.e., the removed code has no side effects.

A barrel file is, mechanically, just a module whose entire body is re-export statements. When a consumer writes:

```
import { ButtonComponent } from '@shared/ui';
```

the bundler must first **execute/evaluate** `@shared/ui/index.ts` to resolve what `ButtonComponent` even refers to. If that barrel contains:

```
export * from './button/button.component';
export * from './modal/modal.component';
export * from './data-table/data-table.component'; // has a module-level side effect, e.g.
registers a custom element
```

the bundler has to include (or at least evaluate) the `data-table` module too, *unless* it can statically prove `data-table.component.ts` has zero side effects — which requires either:

1. `"sideEffects": false` declared in the library's `package.json` (a blanket promise: "nothing in this package does anything just from being imported"), or
2. Rollup/esbuild's own side-effect detection being sophisticated enough to see through the barrel (it often

isn't, especially through deep re-export chains, decorators, or Angular's own metadata side effects on classes).

In practice, this is precisely why several major frontend teams (notably documented by Nx, and independently reported by teams at companies using large Angular monorepos) measured **materially smaller production bundles after flattening or removing barrel files**, or after annotating packages with `sideEffects: false` and using targeted deep imports instead of barrel imports for large shared libraries. The Angular CLI's own esbuild-based builder (default since v17) has improved side-effect analysis over the old webpack builder, but it does not eliminate the fundamental issue: **a barrel is an opacity layer that makes "did importing X also pull in Y and Z" harder for the bundler (and for a human) to answer.**

4.3 How Barrel Files Cause Circular Imports

Circularity happens when module A's evaluation depends on module B's evaluation, and B (directly or transitively) depends back on A. With hand-written direct imports, this is visible in the two files involved. With barrels, it's often invisible until you draw the graph:

```
features/orders/index.ts      re-exports -> data-access/order.service.ts
data-access/order.service.ts imports      -> features/billing (for a shared invoice
type)
features/billing/index.ts      re-exports -> data-access/billing.service.ts
data-access/billing.service.ts imports      -> features/orders (for a shared "linked
order" type)
```

Neither developer who added their single import thought they were creating a cycle — each only looked at their own file. The barrel aggregation is what turns two individually-reasonable imports into `orders -> billing -> orders`.

At runtime, ES module circular imports don't always throw — but they can leave one side of the cycle with an `undefined` binding if a value (as opposed to a type-only or function declaration) is accessed before its module finishes initializing, because ES modules resolve bindings lazily but initialize eagerly top-to-bottom. This class of bug is notoriously hard to reproduce because it depends on **module evaluation order**, which itself depends on **which file happens to be the entry point** — the exact same circular pair can work fine when app A imports first and break when app B (with a different route/entry ordering) imports first. This is a frequent, real-world root cause behind "it works locally / breaks in CI" or "works in dev-server / breaks in prod build" bugs tied to folder/module reorganization.

5. Edge Cases & Gotchas

- **Barrel-induced circular dependency between sibling features.** As shown in §4.3 — the fix is to extract the genuinely-shared type (`LinkedOrderRef`) into a `shared/platform` library that both features depend on downward, restoring a one-directional graph. This is the single most common structural bug found by `madge --circular` or Nx's `dep-graph` visualization in large Angular repos.
- **`shared/` becoming a dumping ground.** Once `shared/` exists, it silently invites anything "kind of reusable" — a `user-profile-card.component.ts` that technically only appears in one feature today gets placed in `shared` "in case it's needed elsewhere," and six months later `shared` has 200 components, half used exactly once, with no clear ownership and a name that no longer describes anything. The discipline: a component moves *into* `shared` only when a **second** feature actually needs it

(rule of three is even safer), not preemptively. Regularly audit `shared` for single-consumer exports and move them back into the one feature that uses them.

- **Over-nested folders hurting discoverability.**

`features/orders/ui/components/detail/sections/header/order-detail-header.component.ts` is a real pattern seen in codebases that took "colocate everything" too literally — six directories deep for one file. Beyond 3–4 levels of nesting under a feature root, developers start relying entirely on fuzzy-file-search and stop being able to reason about the tree visually, which defeats the entire point of a folder structure. A flatter `features/orders/ui/order-detail-header.component.ts` with descriptive filenames scales better than deep nesting with generic ones.

- **Premature feature-splitting.** Creating `features/order-list/`, `features/order-detail/`, `features/order-filters/` as three separate top-level features when they are really three views of one "orders" domain fragments a cohesive concern and multiplies the number of `index.ts` barrels, route files, and lint-boundary configs for no benefit — and makes the genuinely-shared `ordersService` awkward to place (which of the three "owns" it?). Split a feature into more than one top-level folder only when it will realistically be owned by a different team or lazy-loaded independently at a different point in the user journey — not just because the file count within it feels large.
- **`core/` accidentally imported by multiple lazy chunks.** If `core` services are provided via `providedIn: 'root'` this is a non-issue (Angular's DI + tree-shakable providers handle singleton-ness correctly regardless of import path). But if a team instead re-exports `core` internals through a barrel that a lazy-loaded feature imports directly (instead of only relying on DI injection), the *code* for those core services can get duplicated across chunks by the bundler, even though the *DI instance* remains a correct singleton — a subtle bundle-bloat bug that's easy to miss because functionality still works correctly; only bundle-size analysis reveals it.
- **Feature `index.ts` silently widening over time.** A barrel that starts with two intentional exports accumulates a third, a fourth, an internal component "just for this one edge case," and a year later the "public API" is indistinguishable from "everything in the folder" — the discipline erodes without a lint rule or PR-template checklist actively pushing back (e.g., an ESLint rule flagging any export from a feature's `index.ts` that isn't in an explicit allow-list, or a required "why is this now public?" line in the PR description).
- **Renaming a barrel-exported symbol breaks more callers than expected.** Because barrels centralize the "stable" import path, a rename inside the barrel (`OrderSummary` -> `OrderOverview`) still ripples to every consumer — barrels reduce *path* churn, not *API* churn. Teams sometimes mistakenly believe barrels give them free rename-safety; they only give import-path stability, not full encapsulation from renames of what's actually exported.

6. Interview Questions & Answers

Q1: What's the difference between type-based and feature-based folder organization?

Type-based groups files by technical kind (`components/`, `services/`, `models/`) regardless of business domain; feature-based groups files by business capability (`features/orders/`, `features/billing/`) regardless of file type. Type-based is simpler for small apps but forces developers to jump across multiple top-level folders to work on one feature as the app grows; feature-based keeps everything needed for one capability colocated, which scales much better with app size and team size.

Q2: Why do most large Angular apps adopt a `core/shared/features` split specifically?

Interviewer intent: checking whether the candidate understands *why* this specific three-way split exists, not just that it's conventional.

`core` isolates true app-wide singletons (auth, HTTP interceptors, global error handling) that should be instantiated exactly once and are irrelevant to any single feature. `shared` isolates presentation-only, domain-agnostic reusable building blocks (buttons, pipes, generic utilities) with zero coupling to any feature's business model. `features` isolates business-domain-specific code, one folder per capability. The split encodes a dependency direction — features depend on core and shared, never the reverse, and shared never depends on features — which keeps the dependency graph acyclic and each piece independently reasonable about.

Q3: What should live in `core` vs `shared`, and how would you catch a violation in code review?

`core`: singleton services (auth, logging, global HTTP interceptors), app initializers, root guards — things meant to exist exactly once. `shared`: reusable UI components, pipes, directives, and pure utilities with no dependency on any specific feature's domain types. A violation to watch for: a `shared` component importing a type from `features/*` (it isn't actually generic anymore) or a service being duplicated between `core` and a feature because someone didn't realize `core` already provides it. Concretely, in review I'd check whether the "shared" component's `@Input()` types are primitives/generic DTOs versus feature-specific domain models — the latter means it belongs in the feature, not `shared`.

Q4: What is a barrel file, and why would a team choose to use one?

A barrel file (conventionally `index.ts`) re-exports selected symbols from other files in a directory so consumers can import from one aggregated path instead of several deep paths. Teams use them to present a curated public API per feature/library (hiding internal components/services), to keep import paths stable across internal refactors, and to make import statements cleaner for consumers of shared UI libraries.

Q5: What are the concrete downsides of barrel files, and when would you avoid them?

Interviewer intent: distinguishing candidates who've only heard "barrels are good practice" from ones who've actually hit the tree-shaking/circularity problems in production.

Barrels can defeat tree-shaking because bundlers must evaluate the whole barrel module (and prove side-effect-freedom of everything it re-exports) before excluding unused parts — without `sideEffects: false` and clean per-file exports, importing one component can pull in the entire barrel's transitive graph, inflating bundle size. Barrels also make circular imports easy to introduce accidentally, because the aggregation hides the true module graph — a developer adding one import doesn't see that it creates `orders -> billing -> orders`. I'd avoid nesting barrels multiple layers deep (a barrel of barrels) and would avoid using them at all for large shared UI libraries where bundle size matters most, preferring direct/deep imports there, reserving barrels for a single top-level public-API boundary per feature/library.

Q6: How does a barrel file actually interfere with tree-shaking, mechanically?

Tree-shaking removes exports the bundler can statically prove are unused and side-effect-free. Importing a symbol from a barrel forces the bundler to first resolve/evaluate the barrel module itself; if the barrel re-exports several files via `export *` and the bundler can't prove some of those files are side-effect-free (no `sideEffects: false` declared, or complex re-export chains it can't fully trace), it conservatively keeps more code than necessary rather than risk breaking a side effect a real user depended on. The practical fix is declaring `"sideEffects": false` in the library's `package.json` (or listing specific files with side effects) and keeping barrels shallow so the bundler's static analysis can actually succeed.

Q7: Describe a scenario where barrel files cause a circular dependency, and how you'd fix it.

If `features/orders/index.ts` re-exports a service that imports a type from `features/billing`, and `features/billing/index.ts` re-exports something that imports a type from `features/orders`, you get `orders -> billing -> orders` — a cycle that can manifest as an `undefined` value at runtime depending on module evaluation order, and can differ between dev-server and production builds. The fix is to extract the genuinely cross-cutting type into a `shared` or `platform` library that both features import downward from, restoring a one-directional dependency graph, and to run a circular-dependency checker (e.g., `madge --circular`, or Nx's dependency-graph lint) as a CI gate to prevent recurrence.

Q8: How would you structure an Angular app to scale from a small MVP to an enterprise-size codebase without a disruptive big-bang rewrite?

Start flat/type-based for a genuinely small app; as soon as a second feature area and a second contributor appear, introduce `core/shared/features` with feature folders mapped 1:1 to lazy-loaded routes. As the app and team grow further, split `shared` when it starts serving genuinely unrelated concerns (UI kit vs. cross-cutting data-access), and consider migrating to an Nx (or similar) monorepo only once multiple teams/deployables genuinely need independent library boundaries and enforced ownership — not preemptively. The key is each transition is driven by an observed pain point (files hard to find, unclear ownership, accidental coupling) rather than anticipated future scale, and each transition (type-based -> feature-based -> monorepo) is incremental, not a rewrite, because feature folders are already close to what a library becomes.

Q9: What does "public API surface per feature" mean, and how do you enforce it beyond convention?

It means a feature folder deliberately exposes only a small curated set of symbols (typically its route config and a couple of DTO types) via its `index.ts`, while its internal components, services, and state stores stay unexported and inaccessible to the rest of the app — analogous to encapsulation in OOP. Convention alone (code review) tends to erode under deadline pressure, so the durable enforcement mechanism is tooling: an ESLint rule (`no-restricted-imports` blocking deep paths like `features/*/ui/**`, or Nx's `enforce-module-boundaries` keyed on library tags) that fails the build/lint if anything imports past the feature's declared public entry point.

Q10: In an Nx-style monorepo, how do `apps/` and `libs/` differ, and how are library boundaries typically tagged?

`apps/` contains deployable applications (the things that actually get built and shipped, e.g., `storefront`, `admin-console`) with minimal code of their own — mostly composition (routing, bootstrap config) that pulls in libraries. `libs/` contains all reusable code, organized by both `scope` (business domain: `orders`, `billing`, `shared`, `platform`) and `type` (`feature`, `ui`, `data-access`, `util`). Each library declares tags (e.g., `scope:orders`, `type:data-access`) in its project config, and an ESLint rule (`@nx/enforce-module-boundaries`) encodes allowed dependency directions between tags — e.g., a `ui` library may depend only on `util`, a `feature` library may depend on `ui/data-access/util` within its own scope plus `shared/platform`, but never directly on another domain's internals.

Q11: A junior developer asks why you don't just put every reusable component in `shared` "to be safe." How do you respond?

Interviewer intent: probing whether the candidate has actually seen `shared` folder rot happen and has a concrete heuristic to prevent it, versus just repeating "keep it DRY."

I'd explain that "might be reusable someday" isn't the same as "is reused," and that `shared` folders that accept speculative additions become dumping grounds — hundreds of components, most used by exactly one feature, with no clear ownership and no name that still describes their contents. The heuristic I use: a component moves into `shared` only when a second (or per "rule of three," a third) feature genuinely needs it,

not when it's first written. Until then it stays colocated in the feature that uses it — moving it later, once real reuse is proven, is a cheap refactor (a folder move plus an import path change), while un-inlining a wrongly-shared component that turns out to need feature-specific behavior later is a much messier one, because other consumers may already depend on its current generic shape.

Q12: How do you decide when a single feature folder should be split into two separate top-level features?

The signal isn't file count inside the folder — it's ownership and independent lazy-loading value. Split when a different team will realistically own the new piece, or when it genuinely benefits from being loaded independently at a different point in the user journey (e.g., a rarely-visited admin-only sub-area of a customer-facing feature). Splitting purely because a folder "feels big" without either of those drivers just multiplies barrels, route files, and lint-boundary configuration, and creates awkward questions about which of the new folders owns any genuinely shared service between them — a classic premature-abstraction trap in folder form.

7. Quick Revision Cheat Sheet

- **Type-based** (`components/`, `services/`): fine for small apps; forces jumping across folders per feature as the app grows.
- **Feature-based** (`features/orders/`): groups by business domain; scales better; maps naturally to lazy-loaded route chunks and team ownership.
- `core/`: app-wide singletons (auth, interceptors, global error handling); provided once; never re-imported per feature.
- `shared/`: reusable, domain-agnostic, presentation-only building blocks; must never depend on any `features/*` type; add things only after real (2nd/3rd) reuse, not speculatively.
- `features/`: one folder per business capability; ideally 1:1 with a lazy-loaded route; sub-organize into `ui/` and `data-access/`.
- **Dependency direction**: `features -> shared/core`, never `shared -> features`, never `feature A -> feature B` internals.
- **Colocate** component/style/template/spec files in one folder — never a parallel `tests/` tree.
- **Public API per feature**: expose only route config + essential DTO types via `index.ts`; hide internal components/services; enforce with ESLint `no-restricted-imports` or Nx `enforce-module-boundaries`, not just convention.
- **Barrel files**: stabilize import paths and curate public API, but can defeat tree-shaking (bundler must prove side-effect-freedom of everything re-exported) and make circular imports easy to introduce invisibly. Use one barrel per feature/library public boundary; avoid barrels-of-barrels; declare `sideEffects: false` where true.
- **Circular imports via barrels**: fix by extracting the shared type into a `shared/platform` library both sides depend on downward; catch with `madge --circular` or Nx dep-graph in CI.
- **Scaling path**: flat -> `core/shared/features` -> Nx-style `apps/+libs/` monorepo with tagged module boundaries — each step triggered by observed pain (discoverability, ownership, coupling), not anticipated future scale.
- **Gotchas**: shared folder becoming a dumping ground; over-nesting past 3–4 levels killing discoverability;

premature feature-splitting without a real ownership/lazy-load driver; core services accidentally bundled twice via deep imports bypassing DI.

Created By - Durgesh Singh